



AP

Assignment 1

Anders Hoeck, Marcus Frehr Friis-Hansen

September 2026

1 Introduction

The goal of this assignment was to extend the interpreter from the exercises with support for functions, function application, and error handling. The implementation was straightforward but some parts required interpretation in which we used LLM's for better understanding. LLM's has also been used for reasoning about code, debugging, syntax errors etc. For these purposes we used ChatGPT and Claude.

We consider our solution to be functional. Functionality is covered by tests for ForLoop, Lambda, Apply and TryCatch. Each tests ensures that the implementations works and tests edge cases.

All test cases currently pass, therefore we consider the solution to be working as intended. There may still be edge cases not covered but based on the required functionality and test results we consider the quality of our solution to be good.

2 Numbered Questions

2.1 What is the observable evaluation order for Apply, what difference does it make, and which of your test(s) demonstrate this

First Apply evaluates **e1**, meaning the first expression, and then it evaluates **e2**. The evaluation order matters for non-commutative operations such as division. Another effect could be in the case of failure. If **e1** fails, then **e2** is never evaluated. In such a case, the evaluation order is observable in terms of which error message is printed.

our "Apply order" test, shows the importance of the order. Since CstInt 5 causes an "Invalid application" error the second argument never gets evaluated. The second argument would have been a "division by zero" error, but since **e1** is evaluated before **e2** this error never prompts.

2.2 Would it be a correct implementation of eval for TryCatch to first call eval on both sub-expressions? Why or why not? If not, under which assumptions might it be a correct implementation?

No it would not be a correct implementation. If **e1** were to succeed, then **e1** would be the result of the TryCatch expression, it would therefore be unnecessary to also evaluate **e2**. There is also the case where **e2** never terminates, which even though **e1** would terminate, would ruin the whole operation, as **e1** would never be returned.

Under the assumption that **e2** always terminates, the implementation would be correct, just potentially wasteful and unnecessary

2.3 Is it possible for your implementation of eval to go into an infinite loop for some inputs?

The implementations in eval that makes sense to look at is the ones that repeat evaluation. For instance, our ForLoop has a counter the increases by 1 for every iteration. If n is a normal integer it will eventually reach counter $\geq n$ and then the loop reaches *Else Right p Val* and therefore cannot continue forever.

Another one to look at is Apply to see if a function can repeat itself indefinitely. It can potentially go into an infinite loop because evaluating the function body can lead to another apply. If a function is able to call itself repeatedly without reaching a terminating case, eval will continue to evaluate the body forever.

However our Let implementation does not allow this because the variable is added to the environment only after **e1** has been evaluated. Therefore, our implementation of eval should never go into an infinite loop for any input

2.4 For each of the following equalities, explain under which circumstances they hold.

Apply (Lambda "v" x) y == x: This equality only holds when the parameter 'v' does not appear in the body 'x'. In such cases the function is constant, and an evaluation of the function is equal to the body, which is equal to 'x', no matter the value used for the parameter v. In the case where the parameter body 'x' contains 'v', e.g, $x = v + 1$ the equality does not hold, as the function evaluates to a value different than its parameter value.

TryCatch (TryCatch x y) z == TryCatch x y: The equality holds when x OR y evaluates successfully. This equality does not hold only when both x and y fails, and z is evaluated.

2.5 Suppose we wanted APL to be dynamically scoped. What would need to change in your implementation?

Currently when an instance of lambda is evaluated, it takes a copy of the current environment and stores it in ValFun under Env for later use. If we wanted APL to be dynamically scoped we could remove the environment from ValFun. Then we would alter Apply, such that instead of extending the funEnv, we would extend the env from the evaluation of Apply. This would have the effect of using the 'newest' state of the environment from when Apply is evaluated, and not the version, created when lambda was evaluated, making the APL dynamically scoped.

2.6 What is the asymptotic complexity of printExp on a left-nested expression such as Add (Add (Add ...) ...) ...?

We implemented printExp for add like so:

```
printExp (Add e1 e2) = printExp e1 ++ " + " ++ printExp e2
```

If we were to use it on a left nested add-expression of n depth, the cost of concatenating each add-expression would grow proportionally with the length of the string left. This is caused by the fact that in haskell, strings are essentially lists of char's. Concatenating something to the end of a list requires traversing the left list, making the cost proportional to the length of the left list. Summing this growing cost for each additional 'add' approximately equates to

$$1 + 2 + 3 + 4 + \dots + n = \frac{n(n+1)}{2} = \frac{n^2 + n}{2} = O(n^2)$$

2.7 State properties that it would be natural for print Exp to satisfy (you are encouraged to hypothesize additional machinery in order to state such properties).

- PrintExp always finishes and prints a string
- PrintExp is deterministic, if given the same input, it will return the same output
- Let us say we had a parser, that could take the printed string and return back an expression. It would be a naturally beneficial property that the produced expression was the same one used to print the string. Essentially that no information is lost.

2.8 Give a reason for why Val is defined in APL.Eval rather than in APL.AST

We initially found that it would make more sense for Val to be defined in APL.AST, such that the primary type definitions were assembled in a single module. We later considered it beneficial for Val to be defined in APL.Eval as it represents the result of evaluating an expression. This separates the two files from each other semantically, such that APL.AST is the module that defines the program input, and APL.Eval defines the program functionality/results, as Val is only ever used as the result of using Eval.